XHSC 小华半导体 XIAOHUA SEMICONDUCTOR

32-bit Microcontrollers

HC32L072 / HC32L073 / HC32F072 Series

USB SRAM Access Considerations

Applicable objects

L Series	**HC32L072**	**HC32L073**
F Series	**HC32F072**	

Note: Some models do not support USB function, please refer to the data sheet for details.

Table of contents

1 Summary

This document mainly describes the address range, precautions and how to make full use of RAM space to meet the user's development requirements for USB access to RAM in the HC32L072 / HC32L073 / HC32F072 series.

This application note includes:

- Function introduction
- User-defined variable address setting in USB application

Note:

- This application note is an application supplement for the HC32L072 / HC32L073 / HC32F072 series and cannot replace the user manual. Please refer to the user manual for specific functions, register operations and other related matters.

2 Functional Introduction

The address range of the RAM of the HC32L072 / HC32L073 / HC32F072 series in the system map is shown in Table 2-1.

Table 2-1 RAM address mapping range and corresponding capacity

Address range	Capacity	Memory type
0x20000000—0x20003FFF	16 KByte	SRAM

For the HC32L072 / HC32L073 / HC32F072 series USB, the address range for accessing RAM is the first 8K of RAM, and the address range is shown in Table 2-2.

Table 2-2 Address mapping range and capacity of USB accessing RAM

Address range	Capacity	Memory type
0x20000000—0x20001FFF	8 KByte	SRAM

For a detailed introduction to the RAM of the HC32L072 / HC32L073 / HC32F072 series, refer to the RAM Controller chapter in the User Manual.

3 User-defined variable address setting in USB application

The range of USB access to RAM is 0x20000000—0x20001FFF. If it exceeds this range, USB will not be accessible, and USB-related applications may not run normally. Therefore, users must first ensure that USB-related variables are within the address range in RAM. For the arrays or structures sent and received, their starting address and the address of the last byte are also required to be within this range.

At present, for variables in user programs, in order to prevent the USB access range from exceeding its accessible range, the starting address of the user program variables can be defined. For example, when a user defines a large array and the compiler automatically assigns addresses to it, the assigned address may be at the front, and when the size of the array is large enough, the variable accessed by the USB may be outside 0x20000000—0x20001FFF. In this case, even if the compilation is fine, the USB will not work properly when the program is running. In this case, the starting address of the array can be manually defined and placed after the RAM to ensure the normal operation of the USB.

If there are many variables in the user program and the sizes of each variable are uncertain, the user can modify the scattered loading file of the project file to define the address range of the user-defined variables in RAM through the scattered loading file to ensure that the entire project can work properly. The following example implements the range limitation of user variables through the scattered loading file.

For usb_customhid in the USB example, if you want to put all the variables defined in main into the space in RAM starting at 0x20001500, you can write a scatter loading file according to Figure 3-1. In the figure, except for the variable definition range in main.c set to 9472 bytes starting at 0x20001500, the variables defined in other files are all within the 8192 bytes starting at 0x20000000.

```
; *************************************************************
; *** Scatter-Loading Description File generated by uVision ***
; *************************************************************

LR_IROM1 0x00000000 0x00020000  {    ; load region size_region
  ER_IROM1 0x00000000 0x00020000  {  ; load address = execution address
   *.o (RESET, +First)
   *(InRoot$$Sections)
   .ANY (+RO)
   .ANY (+XO)
  }
  RW_IRAM1 0x20000000 0x00002000  {  ; RW data
     .ANY (+RW +ZI)
  }
    RW_IRAM2  0x20001500 0x00002500{
      main.o(+RW, +ZI)
    }
}
```

Figure 3-1 Scatter loading file

Before using the scatter-loading file, first open the "Options for Target 'usbd_customhid'"->Linker page in the keil compiler, as shown in Figure 3-2, uncheck the checkbox circled by the red box, and then select the corresponding scatter-loading file in the "Scatter File" edit box. The settings of this example are shown in Figure 3-2.

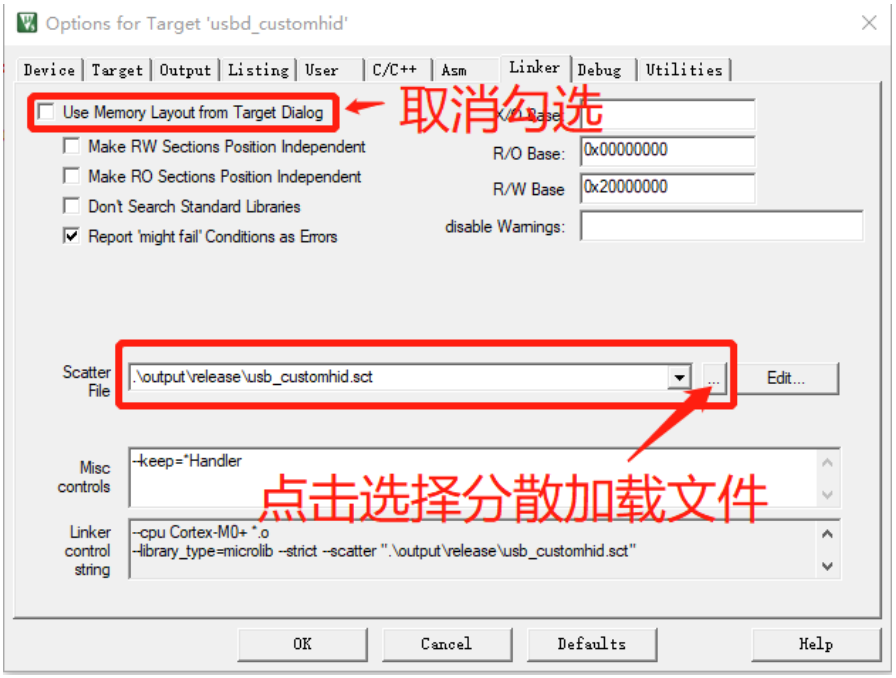

Figure 3-2 Setting interface of using the edited scatter loading file

4 Other information

Technical support information: http://www.xhsc.com.cn

5 Version information & contact information

Date	Version	Change History
2020/10/16	Rev1.0	First version released.
2022/7/15	Rev1.1	Company Logo updated.

If you have any comments or suggestions during the purchase and use process, please feel free to contact us.

Email：mcu@xhsc.com.cn

Website: http://www.xhsc.com.cn

Address: 10th Floor, Building A, No. 1867 Zhongke Road, Pudong New District, Shanghai

Postal Code: 201203